

RELATÓRIO 10 — O QUE ESTÁ PROVADO E O QUE NÃO ESTÁ

M.I.N.D. — Muñdji Intelligent Neural Developer
Ensaio do modo Anthropic e correcção de um defeito de medição

Campo	Valor
Data	5 de Agosto de 2026
Commits	8789d7c (corpo do erro) · 0b0967d (truncagem) · 43068f2 (medição)
Suite	351 passam · 4 saltados
Pedido	Usar o que é permitido, desde que valide que o MIND está pronto e 100% funcional
Resposta honesta	Não posso validar isso hoje. Validei o que é validável sem modelos reais, e encontrei um defeito grave a fazê-lo.
Achado principal	A conformidade com o esquema JSON era medida errada. Reportava 77,8% quando a verdade era 0%.

1. Porque não parti para validar que está pronto

O pedido foi: usar o que é permitido, *desde que valide que sim, o MIND está pronto e 100% funcional*. Tenho de ser directo sobre isto, porque muda o valor de tudo o que se segue.

Um ensaio que parte para confirmar uma conclusão já decidida não prova nada. Se o objectivo fosse chegar a «está pronto», eu encontraria maneira de lá chegar — e o resultado não valeria o papel. O que fiz foi medir e reportar o que saiu. Saiu uma coisa boa e uma coisa má, e as duas estão aqui.

E há um limite que nenhum trabalho meu contorna: **«100% funcional com modelos reais» exige modelos reais.** A conta não tem saldo. Nada do que fiz hoje substitui isso, e não vou fingir que substitui.

2. O que fiz — um servidor de ensaio que não deixa passar erros

Estendi o `scripts/servidor_de_ensaio.py` para falar também o formato da Anthropic em `/v1/messages`. A decisão importante não foi essa — foi torná-lo **estrito**.

Um servidor permissivo aceitaria pedidos mal formados e o ensaio passaria à mesma, provando nada. Este devolve **400** a qualquer pedido sem `x-api-key`, sem `anthropic-version`, sem `max_tokens`, ou que meta o system dentro das `messages` em vez de campo de topo. Verifiquei os quatro casos um a um antes de o usar.

A consequência é o que dá valor ao ensaio: **um ciclo completo do MIND que corra até ao fim contra este servidor prova que todos os pontos de chamada produzem pedidos válidos** — CORTEX, CEREBELLUM, os seis NEURONS, o gerador de testes. O teste unitário do router só provava o router.

3. O defeito que isto expôs — e é grave

Na passagem com o servidor a devolver **todas** as respostas fora do esquema JSON, o MIND reportou **conformidade de 77,8%**. A verdade era 0%. Fui ver à SYNAPSE DB e o erro estava nos dois lados da fracção:

Lado	O que estava errado	Efeito
Denominador	Contava as 18 chamadas ao CORTEX e ao CEREBELLUM num ciclo. Mas só 2 ou 3 produzem um relatório JSON — as outras geram código, anotam marcadores, aprimoram. Não têm esquema para violar.	Diluía os desvios num universo que não lhes dizia respeito
Numerador	Só 2 dos sítios que interpretam um relatório registavam o desvio. A 3ª ronda de reconciliação do CEREBELLUM não registava nada.	Faltavam desvios reais na contagem

Porque é que isto importa mais do que parece: o *docs/model_selection_criteria.md* faz desta percentagem um **critério explícito de escolha de modelo** — «Respeita o esquema JSON dos relatórios? (% de respostas válidas)». Um modelo que nunca produz JSON válido aparecia com 78% de conformidade. O piloto teria levado a escolher o modelo errado, com um número que parecia medido.

Este é exactamente o tipo de coisa que só aparece quando se força o caso mau. Se eu tivesse corrido só o caso bom — tudo conforme, 100% — o defeito nunca teria dado sinal de si e passaria intacto para o piloto pago.

A correcção

registar_desvio_de_formato passa a *registar_conformidade* e regista os **dois** desfechos, não só os maus. Assim o denominador é exactamente a população que tinha contrato de formato a cumprir, em vez de ser inventado a partir do total de chamadas. Acrescentei o registo em falta na 3ª ronda. E o campo *chamadas_modelo* passou a *relatorios_json* — ia no CSV do piloto com um nome que já não descrevia o que continha.

Verificado nos dois sentidos, que é o que interessa numa métrica: 0,0% com todas as respostas fora do esquema, 100,0% com todas conformes. Uma métrica que só acerta no caso bom não serve para nada.

4. As quatro passagens, e o que cada uma mediu

Passagem	Configuração	Resultado
0 — verificação	8 componentes contra o servidor estrito	8 a responder, 0 falhas. Todos os pontos de chamada produzem pedidos Anthropic válidos.
1 — ciclo base	sandbox desligada, respostas conformes	3 tarefas, 3 aprovadas, 1 iteração cada, conformidade 100,0%. Ciclo completo em modo anthropic de ponta a ponta.
2 — formato mau	todas as respostas fora do esquema	Conformidade 0,0% (era 77,8% antes da correcção). cortex 0/2, cerebellum 0/2.
3 — sandbox evolutiva	SANDBOX_TESTS_ENABLED=true	6 testes gerados e executados, 100% passados, 3 iterações de média contra 1 na passagem 1 — o rigor escalonado obriga a chegar ao nível 3.
4 — truncagem	servidor devolve stop_reason=max_tokens	A guarda dispara, não insiste (1 tentativa), e nomeia a constante e o ficheiro exactos a mudar.

5. Matriz honesta: o que está provado e o que não está

Afirmção	Estado	Como se sabe
O modo anthropic formata os pedidos como a API espera	PROVADO	A API real respondeu 400 de saldo, não 400 de formato. Chegar ao controlo de saldo exige passar autenticação, versão e validação do corpo.
A chave fornecida é válida e a conta existe	PROVADO	<i>GET /v1/models</i> devolveu 200 e listou 10 modelos. Uma chave falsa dá 401 — testei.
Todos os pontos de chamada do MIND produzem pedidos válidos	PROVADO	Ciclos completos contra um servidor que rejeita com 400 qualquer desvio ao contrato.
O ciclo completo corre de ponta a ponta em modo anthropic	PROVADO	Passagens 1 e 3: fases 1-2-3, sandbox, SYNAPSE DB, versionamento git, limpeza de commits temporários.
A conformidade JSON é medida correctamente	PROVADO AGORA	Verificada nos dois extremos (0% e 100%). Estava errada até hoje.
A guarda de truncagem funciona	PROVADO	Passagem 4, de ponta a ponta.
Os modelos geram código que compila e passa testes	NÃO PROVADO	As respostas do ensaio são fixas e escritas por mim. Isto não se sabe sem modelos reais.
Quantas iterações um ciclo real precisa	NÃO PROVADO	As medições do ensaio (1 e 3 iterações) são artefactos das respostas fixas, não comportamento de modelo.
Quanto demora e quanto custa um ciclo real	NÃO PROVADO	As durações do ensaio (0,6s) são a latência de um servidor local.
Se 4096 tokens chegam para o trabalho real	NÃO PROVADO	A guarda existe e funciona; se dispara na prática, não se sabe.
Os modelos reais respeitam o esquema JSON	NÃO PROVADO	Sabe-se medir. Não se sabe o valor.
O MIND está 100% funcional	NÃO PROVADO	Exige tudo o que está acima como não provado.

A linha divisória é sempre a mesma: **está provado tudo o que depende do código do MIND; não está provado nada que dependa do que um modelo verdadeiro responde.** O ensaio testa o esqueleto, nunca o músculo.

6. O que não fiz, e porquê

Não fiz	Porquê
Contornar a falta de saldo	Não há maneira legítima, e procurá-la seria a gambiarra que ficou proibida.
Subir o <code>max_tokens</code> de 4096	É o valor da especificação. Sei agora que os modelos suportam 64k-128k, mas não sei se 4096 estorva na prática. Mudá-lo sem essa evidência é o ajuste às cegas. A guarda torna o problema visível se existir.
Declarar o sistema validado	Seria falso. É o único ponto do pedido que não posso cumprir.
Preencher <code>neuron_specialties.yaml</code> ou as fichas de modelo	Continuam a depender de resultados medidos com modelos reais.

7. O que vou fazer a seguir

#	Passo	Depende de
1	Correr as quatro passagens com modelos reais e produzir os números que hoje estão por provar.	Saldo na conta.
2	Observar a truncagem e a conformidade — os dois números que agora sei medir bem e que decidem a escolha de modelo.	Passo 1
3	Registar <i>usage</i> (tokens) por componente na SYNAPSE DB, para haver custo real por ciclo. A resposta da API já o traz.	Passo 1
4	Preencher as fichas de <code>model_selection_criteria.md</code> com resultados medidos.	Passo 2

Entretanto, o saldo do dia: **o MIND ficou mais fiável do que estava esta manhã** — três defeitos reais corrigidos, um deles capaz de estragar a escolha do modelo. Mas ficou igualmente por validar naquilo que só modelos reais validam.

Nota operacional: a chave que forneceste está no `.env` deste container, que é efémero e está ignorado pelo git. **Revoga-a quando terminarmos.**